

Annex II — Relational Automation, Graph Routing, and Gateway Mechanics

BEAP™ / WR Desk™ Canonical Specification — standalone Annex. Renumbered from the earlier I.13–I.17 draft to avoid collision with existing Annex I sections. Mapping: I.13 → II.1 · I.14 → II.2 · I.15 → II.3 · I.16 → II.4 · I.17 → II.5. References of the form "Annex I §I.x" point to the existing *Annex I — Protocol Enforcement Mechanics*.

II.0 Status, Scope, and Relationship to the Canon

This Annex is normative unless explicitly marked as informative.

It defines the deployment scope of BEAP™, the routing and lifecycle semantics of the WR Graph™, the permission-bearing asymmetric handshake and the BEAP™ Gateway, feedback reintegration, and the WCR authoring lifecycle.

This Annex does not redefine or extend wire formats, capsule serialization, cryptographic primitives, transport mechanisms, or PoAE™ execution semantics; these remain exclusively defined in the Canon and its Normative Annex. All enforcement mechanics of Annex I apply unchanged; nothing in this Annex weakens or contradicts them.

II.1 Deployment Scope of BEAP™ (Normative)

II.1.1 Current Deployment Scope

BEAP™ is currently specified, implemented, and validated exclusively for deployment within:

- Optirando™ / WR Desk™ environments
- Silicon Cargo environments

Within this scope, all conformance guarantees of the Canon and its Annexes apply in full.

BEAP™ SHALL NOT be represented, marketed, or relied upon as a general-purpose third-party interoperability standard at this stage. Conformance claims by implementations outside the scope above are undefined.

II.1.2 Future Third-Party Opening (Informative)

Opening BEAP™ to third-party implementers is a long-term objective, not a present property of the protocol. Such an opening would require, at minimum:

- a published conformance test suite,
- capsule and gateway certification procedures,
- a governance process for protocol evolution.

Nothing in this Annex implies a timeline or commitment for this opening. Until such a program exists, references to third-party use SHALL be framed as outlook only.

II.2 WR Graph™ Adapter Activation and Routing Semantics (Normative)

II.2.1 Adapter Bindings as Graph Content

WR Graph™ nodes and subgraphs MAY carry bindings to fine-tuned, LoRA-class domain adapters as first-class context. An adapter binding associates a context region (device class, workflow stage, topic, publisher relationship) with the parametric competence required to reason within it.

II.2.2 The WR Graph™ as Activation Trigger Layer

The WR Graph™ SHALL act as the authoritative trigger layer for adapter lifecycle operations, namely:

- **Activation** — loading an adapter when its bound context becomes active
- **Hot-swap** — replacing the active adapter when context resolution changes
- **Combination** — composing multiple adapters where a context legitimately spans several bound domains
- **Deactivation** — unloading adapters whose bound context is no longer active

Lifecycle operations SHALL be triggered exclusively by deterministic context-resolution events, including:

- WR Code® scan resolution
- workflow state transitions within an active recorded workflow
- topic classification of the active session against the graph
- handshake scope activation (see II.3)

II.2.3 Determinism and Authority Constraints

- Adapter selection SHALL be deterministic: identical context resolution against identical graph state SHALL yield the identical adapter set.
- Adapter activation SHALL NOT expand capability scopes. Capabilities remain exclusively governed by the capability engine, policy engine, and PoAE™ enforcement.
- Every adapter lifecycle event SHALL be logged with context reference, graph version, and adapter identity, and SHALL be auditable.
- Adapters and their weights SHALL remain within the user's trust domain (local Host AI); the graph selects, the host loads.

II.2.4 Multidimensional Graph Structure (Informative)

The WR Graph™ is not a conventional knowledge graph. It comprises multiple interconnected layers over one context space:

1. **Declarative layer** — the structured knowledge graph itself (entities, relations, constraints, decisions)
2. **Episodic layer** — long-term interaction memory bound to graph regions
3. **Parametric layer** — adapter bindings carrying domain competence (II.2.1)

4. **Procedural layer** — embedded Workflow Context Recorder automations (Annex I §I.6, II.5)
5. **Organizational layer** — background sorting, prioritization, and self-optimization of the above, driven by usage, corrections, and authorized decisions

Cross-layer edges enable a single context resolution (e.g. one scan) to select all layers simultaneously: what is known, what has happened, which competence applies, and what can be executed.

Adapters carried in the parametric layer are produced by the adaptation pipeline of Annex I §I.12 (project documentation → coverage-gap identification → curated synthetic datasets → iterative local fine-tuning); the routing and application of that output is demonstrated in the accompanying paper "High-Impact Examples of the Optirando™ System in Action" (§2.3).

II.2.5 Routing Semantics and the Determinism Boundary (Normative)

The WR Graph™'s routing function is **preparatory, never executive**. It selects and proposes — knowledge regions, adapters, and candidate automation paths — but executes nothing. The determinism guarantee of this specification attaches to the **execution artifact**, not to the routing:

- A finalized WCR recording, including its tool invocations, SHALL be deterministic: identical recording version plus identical declared inputs SHALL yield the identical action sequence.
- Consequential steps SHALL be gated by user consent at the applicable PoAE™ tier **regardless of how the request was routed**. No routing outcome can substitute for, weaken, or pre-satisfy an authorization requirement.
- Routing state SHALL be derived exclusively from local, user-authorized signals (usage, ratings, corrections, authorized decisions). External signals — including publisher feedback, incoming capsules, or gateway traffic — SHALL NOT influence routing state.

Consequently, evolution of the routing layer over time affects only which validated options are surfaced, never what an option does when executed or whether it may execute.

II.2.6 Graph Admission Control (Normative)

LoRA-class adapters and WCR recordings SHALL NOT be admitted into the WR Graph™ automatically. Admission requires explicit user consent, consistent with the principle that trust escalation is explicit and never implicit (Annex I §I.7.1). Until admitted, such artifacts remain in their respective libraries or in the sandbox trust domain and are not available to routing.

II.2.7 Inspectability and Snapshots (Normative)

The WR Graph™ SHALL be visually inspectable by the user. The system SHALL support taking an immutable **snapshot** of the graph at any point in time, so that its state remains reviewable even as the graph evolves.

Full graph versioning MAY be implemented; in **High-Assurance Mode** (Annex I §I.8), graph state relevant to routing and automation admission SHALL be versioned, so that any past routing or admission decision can be reconstructed against the graph state under which it was made.

II.2.8 Graceful Degradation and Domain Boundedness (Normative)

Automations are domain-bound execution logic, not a universal interaction mode. If context resolution finds no matching domain region — including the empty-graph case — the system SHALL degrade gracefully:

- no domain adapter is loaded; the normally selected base model answers,
- no automation path is offered,
- no routing is fabricated from insufficient context.

Absence of domain expertise SHALL be surfaced as such, not masked by a low-confidence match.

II.3 Permission-Bearing Asymmetric Handshake and the BEAP™ Gateway

II.3.1 Extension of Handshake Semantics (Normative)

In addition to counterparty identity, declared purpose, scope constraints, and the handshake identifier (H-ID), a handshake MAY carry **automation permission grants**. A permission grant SHALL specify:

- the permitted automation classes or named workflow capsules
- per-step capability ceilings (spend, scope, domain restrictions)
- the PoAE™ tier mapping applicable to each consequential step
- validity period and revocation conditions

Permission grants SHALL be recorded in the relational context graph on the user side **and** in the publisher's backend on the gateway side. Execution SHALL require agreement of both records; any mismatch SHALL fail closed.

Revocation of the handshake SHALL immediately invalidate all future activations under its permission grants, consistent with Annex I §I.4.2.

II.3.2 BEAP™ Gateway (Normative Definition)

A **BEAP™ Gateway** is a server-adapted variant of the orchestrator, embeddable into a website and its backend. The Gateway:

- SHALL issue and accept handshake requests on behalf of the publisher
- SHALL connect to the publisher's database exclusively through a bounded, declared connector
- SHALL publish WCR-recorded automations as validated workflow capsules
- MAY maintain a catalog of such capsules covering the publisher's recurring support and configuration topics (e.g. its FAQ corpus)
- SHALL be operable only by publishers that have completed DNS-based publisher verification — the same verification already required for the publication of public BEAP™ capsules

The Gateway inherits all orchestrator-side enforcement obligations; it does not introduce a weaker conformance class.

II.3.3 Registration-Coupled Handshake (Informative)

In the ordinary registration flow of a website, the user MAY additionally conclude an asymmetric handshake with the publisher through the site's BEAP™ Gateway. Registration and handshake remain separable acts: the handshake adds a governed automation relationship on top of the conventional account relationship. The publisher thereby gains a realtime, structured channel to the user's orchestrator instead of static or after-the-fact data exchange.

II.3.4 Asymmetric Channel Model (Normative)

The relationship established under II.3.1–II.3.3 is deliberately asymmetric:

Forward channel (publisher → user) MAY carry:

- workflow capsules (WCR-recorded automations)
- context blocks (per the Relational Context Graph model)
- status and versioning information

Backchannel (user → publisher) SHALL be restricted to:

- PoAE™ authorization records
- explicitly declared data items covered by the permission grant (e.g. address data)
- status and outcome messages
- user feedback artifacts (screenshot plus textual description), subject to text-purity and attachment-handling rules

The backchannel SHALL NOT carry arbitrary content, telemetry, capsules, or undeclared data classes. This restriction SHALL be enforced fail-closed at the user's orchestrator. The asymmetry is a privacy and containment property, not an implementation convenience.

II.3.5 Execution Model and Publisher-Sanctioned Automation (Normative)

Automations delivered over the forward channel are recorded by the publisher against the publisher's own surfaces (website, dashboard, configuration console) and executed **in the user's environment** against those surfaces.

The decisive governance point: website operators generally do not permit automation on their surfaces. Under a permission-bearing handshake, the publisher **explicitly sanctions** automation on its own surfaces, and because publisher and surface evolve under one governance, recordings and interface remain aligned (cf. §2.3.3 of the Examples paper).

The following invariants remain unmodified:

- Incoming workflow capsules SHALL traverse the full Ingestor–Validator pipeline (Annex I §I.3) and remain sandboxed until validated.
- Capsules SHALL be cryptographically bound to the H-ID and inherit handshake lifetime constraints (Annex I §I.4.2).
- Recorded automations run to the authorization boundary and no further; every consequential step is gated by its PoAE™ tier as mapped in the permission grant.
- The publisher's Gateway is connected to the publisher's own host over an internal handshake, subject to the same enforcement model.

II.3.6 Capsule Catalogs and Graph-Routed Dispatch (Informative)

A publisher will typically author not one automation but a catalog — for example, on the order of a hundred WCR recordings covering the topics of its FAQ, its onboarding flows, and its recurring configuration tasks. Dispatch of a catalog entry to a user is triggered in one of two ways:

1. **Question-routed dispatch.** The user submits a support request in natural language. The WR Graph™'s organizational layer performs the routing: it matches the question against the catalog's semantic annotations so that the correct question triggers the correct recording. The Gateway's server-side orchestrator then transmits exactly that capsule to the user over the P2P connection.
2. **Explicit dispatch.** The user triggers a specific automation directly via a button or link on the publisher's website; the corresponding capsule is transmitted without routing.

In both cases the user's orchestrator receives the capsule through the standard ingestion and validation pipeline, executes it locally under the handshake's permission grants, and the user **rates the outcome** (thumbs up / thumbs down). The rating closes the loop of II.3.7 and, on the publisher side, refines the routing quality of the catalog.

II.3.7 Iterative Support and Feedback Loop (Informative)

Where an automation does not resolve the user's problem, the low-latency P2P relationship enables an iterative loop:

1. Automation runs and fails or terminates incompletely.
2. The user returns a feedback artifact over the backchannel: screenshot plus description ("this did not work").
3. The publisher (or its support tooling) derives a tailored workflow for exactly this state and ships it over the forward channel.
4. The loop iterates until the user confirms resolution ("thumbs up").
5. The resolved session — including the working configuration path — is persisted in the user's WR Graph™, so the graph afterwards holds all details of the successful setup.

Representative use case: fully automated execution of complex third-party configurations (e.g. a Cloudflare setup), where the permission grant covers the configuration steps and any residual failure state is round-tripped as screenshot plus description until a tailored workflow succeeds.

II.4 Feedback Reintegration into the WR Graph™ (Normative)

Outcomes of automation runs, support cases, errors, user actions, and website interactions MAY be reintegrated into the WR Graph™ as versioned context, subject to:

- explicit provenance and temporal ordering (consistent with the append-and-enrich model of Annex I §I.12.2)
- scope binding to the originating relationship, project, or context region
- the organizational layer's background sorting (II.2.4), which assigns reintegrated material to its topic without user effort

Reintegrated feedback SHALL NOT modify capability scopes, permission grants, or enforcement rules; it enriches context only. User feedback MAY, however, cause an automation path to be **superseded or deprecated** within the graph (e.g. after repeated negative ratings or confirmed interface drift), so that routing ceases to surface stale paths; supersession preserves lineage and is driven exclusively by local, user-authorized signals (II.2.5).

Where background optimization proposes a **revision** to an admitted recording — a redundant step removed, a more direct path, an updated constraint — the proposal SHALL NOT replace the active recording silently. Proposed revisions SHALL pass the authoring validation loop (II.5.3) and require explicit user approval; an approved revision replaces its predecessor as an authorized change with recorded rationale and preserved lineage. Over time this yields iterative improvement of the graph per project and per relationship.

II.5 WCR Authoring Lifecycle (Normative)

II.5.1 Authoring Scope and Storage Targets

WCR authoring is not restricted to publishers. Any user MAY author recordings for their own environment; publishers author them for delivery through a BEAP™ Gateway (II.3.6). A confirmed recording is stored in two places:

- the author's **recording library**, where it is versioned and retrievable as an artifact in its own right, and
- where meaningful, the **WR Graph™**, embedded into the appropriate domain region as an executable automation path — assignment is performed automatically by the organizational layer (II.2.4), not by manual filing.

II.5.2 Authoring Paths

A Workflow Context Recorder artifact MAY be authored via two paths:

Path A — Video-derived authoring. The user uploads a video (screen recording, tutorial, or third-party material such as a YouTube video) together with a textual instruction describing the intended workflow. The system derives a candidate semantic manuscript from video and instruction.

Path B — Direct recording. The user performs the workflow while the recorder captures all available signals simultaneously — video, audio, the semantic action sequence, and structural interface metadata — and fuses them into one accurate semantic instruction. No single signal is authoritative; fusion follows the precedence rules of Annex I §I.11.6.

II.5.3 Validation Loop

Authoring SHALL proceed through a staged validation loop:

1. **Candidate manuscript** is generated (Path A or B).
2. **Supervised test run:** the candidate is executed in the sandbox; the semantic layer of the execution is recorded and reconciled against the manuscript.
3. **Final test run:** the reconciled manuscript is executed again for confirmation.
4. **User confirmation:**

- **Thumbs up** → the manuscript is promoted to a stored WCR record and automatically classified into the appropriate topic region of the WR Graph™ by the organizational layer (background sorting, no manual filing).
- **Thumbs down** → the user describes what went wrong; the description enters the manuscript as corrective context, and the loop returns to step 2.

Revisions proposed by background optimization (II.4) SHALL pass through this same loop before replacing an active recording.

II.5.4 Preserved Constraints

All constraints of Annex I §I.6 apply unchanged to both authoring paths: no executable macros, no replay scripts, no binding to static selectors or pixel coordinates; the stored artifact is a structured textual manuscript requiring mediated interpretation under BEAP™ governance.

Video-derived candidates (Path A) SHALL be treated as external-origin artifacts and remain confined to the sandbox trust domain until validated (Annex I §§I.2.2, I.3).

II.5.5 Tool Invocation Constraints (Normative)

WCR recordings encompass tool use, but tool use SHALL be treated as an explicit, separately governed element of the recording — not as an implicit side effect of replay:

- Every tool invocation within a recording SHALL be declared in the manuscript with tool identity and declared inputs.
- Tools SHALL be subject to **explicit policy whitelisting**. Invocation of a non-whitelisted tool SHALL fail closed; whitelisting is a policy-engine decision, never a routing outcome.
- Tool invocations SHALL be routed through the sandbox sub-orchestrator unless a tool is explicitly designated for host execution by policy.
- Tool results SHALL be returned to the calling context in **text format only**, consistent with the text-purity invariant; binary or opaque results remain confined to the sandbox trust domain.
- Once a recording is finalized, its tool invocations are part of the deterministic execution artifact (II.2.5): fixed, inspectable, and cryptographically bound at the point of consequence.

License Notice

This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.